

Linear regression in R

Jeff Leek

- Dependencies
- Download the data
- Fit a simple linear regression
- Is fitting the line a good idea?
- What about categorical variables?
- Visualize the difference between genders
- “dummy” variables
- R’s modeling formulae do this for you automatically
- Peeking “under the hood” of the variables used in the model
- Categorical variables with multiple levels
- This leads to multiple coefficients for one variable
- Adjusting for variables
- Interactions (careful!)
- A few things to be aware of
- Notes and further reading
- Session information

Dependencies

This document depends on the following packages:

```
library(devtools)
library(Biobase)
library(broom)
```

To install these packages you can use the code (or if you are compiling the document, remove the `eval=FALSE` from the chunk.)

```
install.packages(c("devtools", "broom"))
source("http://www.bioconductor.org/biocLite.R")
biocLite(c("Biobase"))
```

Download the data

Here we are going to use some data from the paper Detection of redundant fusion transcripts as biomarkers or disease-specific therapeutic targets in breast cancer. (<http://www.ncbi.nlm.nih.gov/pubmed/22496456>) that uses data from different normal human tissues (called the Illumina BodyMap data).

```

con =url("http://bowtie-bio.sourceforge.net/recount/ExpressionSets/bodymap_eset.RData")
load(file=con)
close(con)
bm = bodymap.eset
pdata=pData(bm)
edata=as.data.frame(exprs(bm))
fdata = fData(bm)
ls()

```

```

## [1] "bm"          "bodymap.eset" "con"          "edata"
## [5] "fdata"       "pdata"        "tropical"

```

Fit a simple linear regression

Here we regress the counts for the first gene on the age of the people from whom the tissue was drawn.

```

edata = as.matrix(edata)
lm1 = lm(edata[1,] ~ pdata$age)
tidy(lm1)

```

```

##           term  estimate std.error statistic    p.value
## 1 (Intercept) 2187.90554 402.998816  5.429062 8.884114e-05
## 2  pdata$age   -23.24999   6.390269 -3.638343 2.686225e-03

```

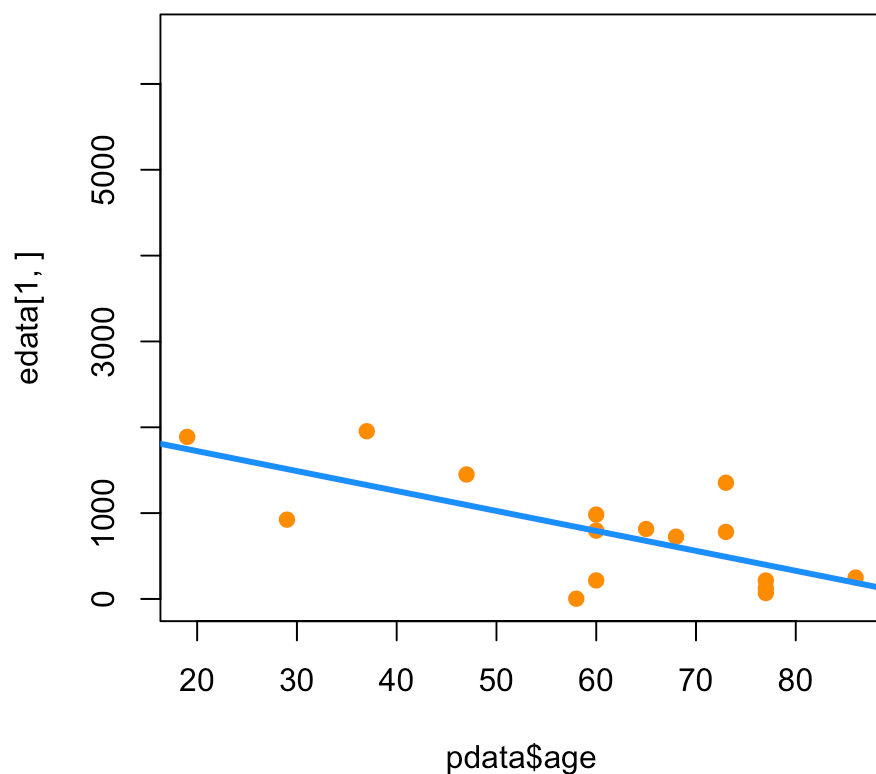
Is fitting the line a good idea?

Visual diagnostics are often some of the most helpful

```

plot(pdata$age,edata[1,], col=1)
abline(lm1$coeff[1],lm1$coeff[2], col=2,lwd=3)

```



What about categorical variables?

```
pdata$gender
```

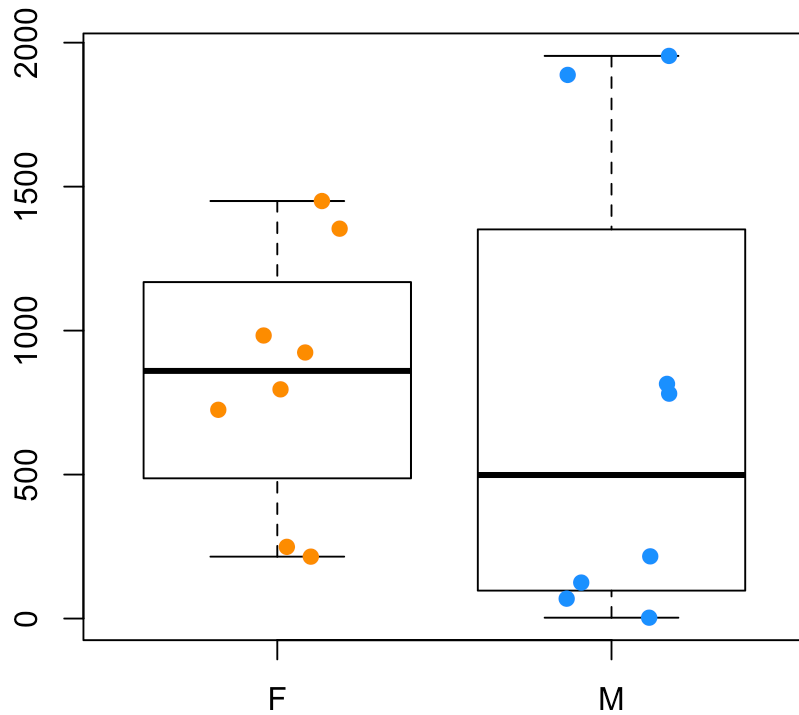
```
## [1] F M F F F M F M M F <NA> <NA> <NA> F
## [15] M M M F M
## Levels: F M
```

```
table(pdata$gender)
```

```
##
## F M
## 8 8
```

Visualize the difference between genders

```
boxplot(edata[1,] ~ pdata$gender)
points(edata[1,] ~ jitter(as.numeric(pdata$gender)),
       col=as.numeric(pdata$gender))
```



“dummy” variables

```
dummy_m = pdata$gender=="M"
dummy_m
```

```
## [1] FALSE TRUE FALSE FALSE FALSE TRUE FALSE TRUE TRUE FALSE NA
## [12] NA NA FALSE TRUE TRUE TRUE FALSE TRUE
```

```
dummy_f = pdata$gender=="F"
dummy_f
```

```
## [1] TRUE FALSE TRUE TRUE TRUE FALSE TRUE FALSE FALSE TRUE NA
## [12] NA NA TRUE FALSE FALSE FALSE TRUE FALSE
```

R’s modeling formulae do this for you automatically

```
lm2 = lm(edata[1,] ~ pdata$gender)
tidy(lm2)
```

```
##           term estimate std.error  statistic    p.value
## 1  (Intercept)  837.000   228.8055   3.6581292 0.002582905
## 2 pdata$genderM -105.625   323.5798  -0.3264264 0.748930374
```

Peeking “under the hood” of the variables used in the model

```
mod2 = model.matrix(~pdata$gender)
mod2
```

```
##      (Intercept) pdata$genderM
## 1             1             0
## 2             1             1
## 3             1             0
## 4             1             0
## 5             1             0
## 6             1             1
## 7             1             0
## 8             1             1
## 9             1             1
## 10            1             0
## 14            1             0
## 15            1             1
## 16            1             1
## 17            1             1
## 18            1             0
## 19            1             1
## attr(,"assign")
## [1] 0 1
## attr(,"contrasts")
## attr(,"contrasts")$`pdata$gender`
## [1] "contr.treatment"
```

Categorical variables with multiple levels

```
table(pdata$tissue.type)
```

```
##
##      adipose      adrenal      brain      breast
##          1          1          1          1
##      colon      heart      kidney      liver
##          1          1          1          1
##      lung      lymphnode      mixture      ovary
##          1          1          3          1
##      prostate skeletal_muscle      testes      thyroid
##          1          1          1          1
## white_blood_cell
##          1
```

```
pdata$tissue.type == "adipose"
```

```
## [1] TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [12] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
```

```
pdata$tissue.type == "adrenal"
```

```
## [1] FALSE TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [12] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
```

This leads to multiple coefficients for one variable

```
tidy(lm(edata[1,] ~ pdata$tissue.type ))
```

```
##           term estimate std.error  statistic
## 1      (Intercept) 1354.000  1017.983  1.33008119
## 2  pdata$tissue.typeadrenal -1138.000  1439.645 -0.79047247
## 3  pdata$tissue.typebrain -1139.000  1439.645 -0.79116709
## 4  pdata$tissue.typebreast -430.000  1439.645 -0.29868468
## 5  pdata$tissue.typecolon -629.000  1439.645 -0.43691317
## 6  pdata$tissue.typeheart -1229.000  1439.645 -0.85368248
## 7  pdata$tissue.typekidney -558.000  1439.645 -0.38759547
## 8  pdata$tissue.typeliver  600.000  1439.645  0.41676932
## 9  pdata$tissue.typelung -539.000  1439.645 -0.37439777
## 10  pdata$tissue.typelymphnode -1105.000  1439.645 -0.76755016
## 11  pdata$tissue.typemixture 4043.333  1175.465  3.43977205
## 12  pdata$tissue.typeovary   96.000  1439.645  0.06668309
## 13  pdata$tissue.typeprostate -573.000  1439.645 -0.39801470
## 14  pdata$tissue.typeskeletal_muscle -1285.000  1439.645 -0.89258095
## 15  pdata$tissue.typetestes  534.000  1439.645  0.37092469
## 16  pdata$tissue.typethyroid -371.000  1439.645 -0.25770236
## 17  pdata$tissue.typewhite_blood_cell -1351.000  1439.645 -0.93842558
##      p.value
## 1  0.31489301
## 2  0.51209555
## 3  0.51176898
## 4  0.79335656
## 5  0.70482162
## 6  0.48321178
## 7  0.73567624
## 8  0.71731923
## 9  0.74407734
## 10 0.52298771
## 11 0.07511726
## 12 0.95290026
## 13 0.72908593
## 14 0.46626631
## 15 0.74629792
## 16 0.82072898
## 17 0.44708910
```

Adjusting for variables

```
lm3 = lm(edata[1,] ~ pdata$age + pdata$gender)
tidy(lm3)
```

```
##           term estimate std.error  statistic    p.value
## 1  (Intercept) 2331.5814  438.181127  5.3210448 0.000138739
## 2    pdata$age  -23.9133    6.488058 -3.6857414 0.002743642
## 3  pdata$genderM -207.2565  236.430512 -0.8766066 0.396610372
```

Interactions (careful!)

You can add interaction terms, but there are a couple of things to keep in mind:

- Interpretation can be challenging
- You should always also include main effects
- Interactions in genomics data are in general hard to detect, and can be fraught for example in gene by environment interactions, the interaction term only tells you that there may be an association, but there are lots of reasons for this statistical association that aren't biological interaction.

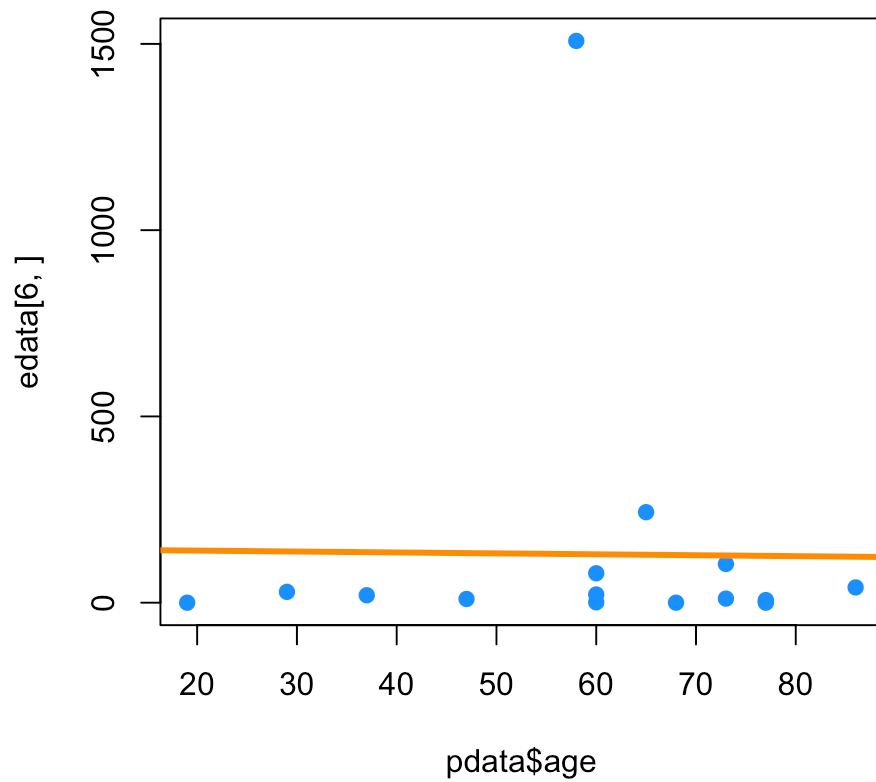
```
lm4 = lm(edata[1,] ~ pdata$age*pdata$gender)
tidy(lm4)
```

```
##               term   estimate std.error statistic    p.value
## 1      (Intercept) 1679.37818 610.165662   2.752332 0.01752641
## 2      pdata$age   -13.47805   9.425164  -1.430007 0.17823223
## 3      pdata$genderM 912.51047 793.378850   1.150157 0.27248454
## 4  pdata$age:pdata$genderM -18.46210 12.536549  -1.472662 0.16658057
```

A few things to be aware of

Outliers can have a big impact on the regression, depending on where they land. Here there isn't much impact

```
lm4 = lm(edata[6,] ~ pdata$age)
plot(pdata$age, edata[6,], col=2)
abline(lm4, col=1, lwd=3)
```

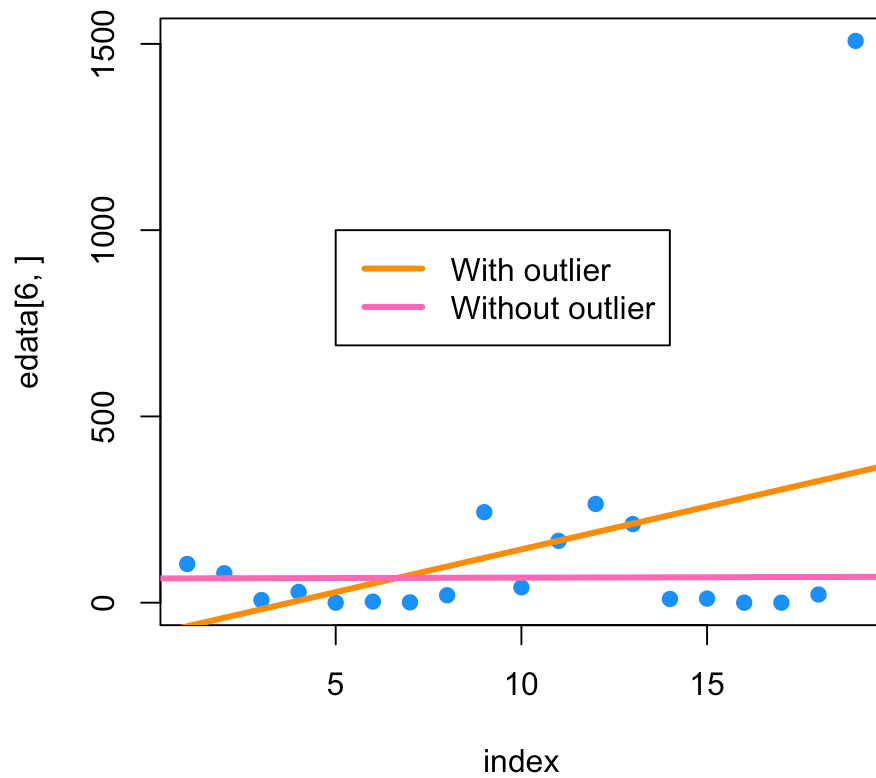



But in this case there is a huge impact

```
index = 1:19
lm5 = lm(edata[6,] ~ index)
plot(index, edata[6,], col=2)
abline(lm5, col=1, lwd=3)

lm6 = lm(edata[6, -19] ~ index[-19])
abline(lm6, col=3, lwd=3)

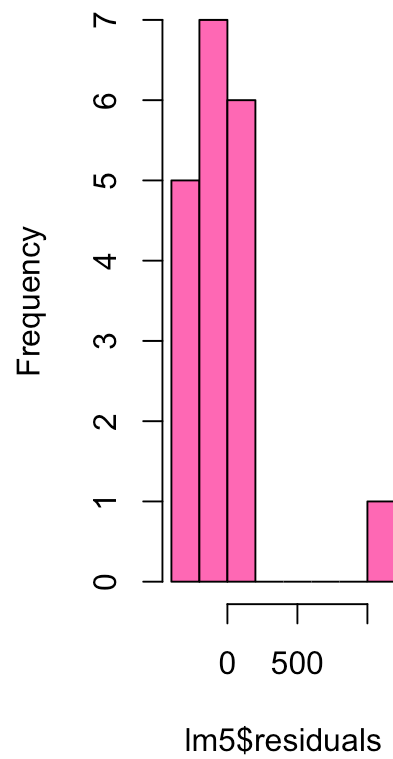
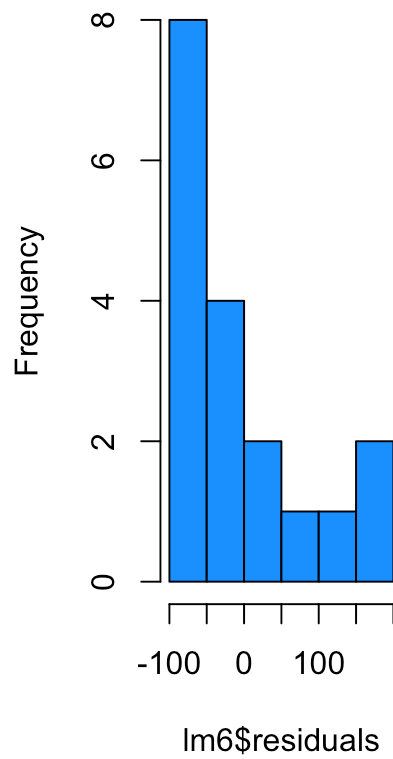
legend(5, 1000, c("With outlier", "Without outlier"), col=c(1, 3), lwd=3)
```



In general you'd like your residuals to look symmetrically (e.g. approximately Normally) distributed but they aren't here. Outliers in the residuals aren't great either.

```
par(mfrow=c(1,2))  
hist(lm6$residuals,col=2)  
hist(lm5$residuals,col=3)
```

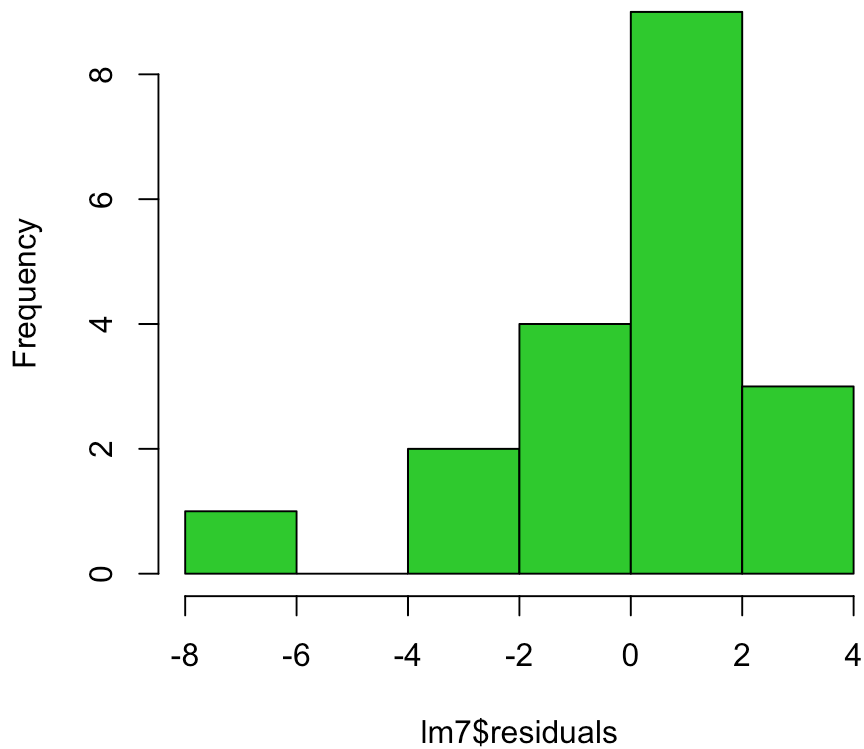
Histogram of lm6\$residuals Histogram of lm5\$residuals



Data transforms are often applied before regression and the residuals look a little better here.

```
gene1 = log2(edata[1,]+1)
lm7 = lm(gene1 ~ index)
hist(lm7$residuals,col=4)
```

Histogram of lm7\$residuals



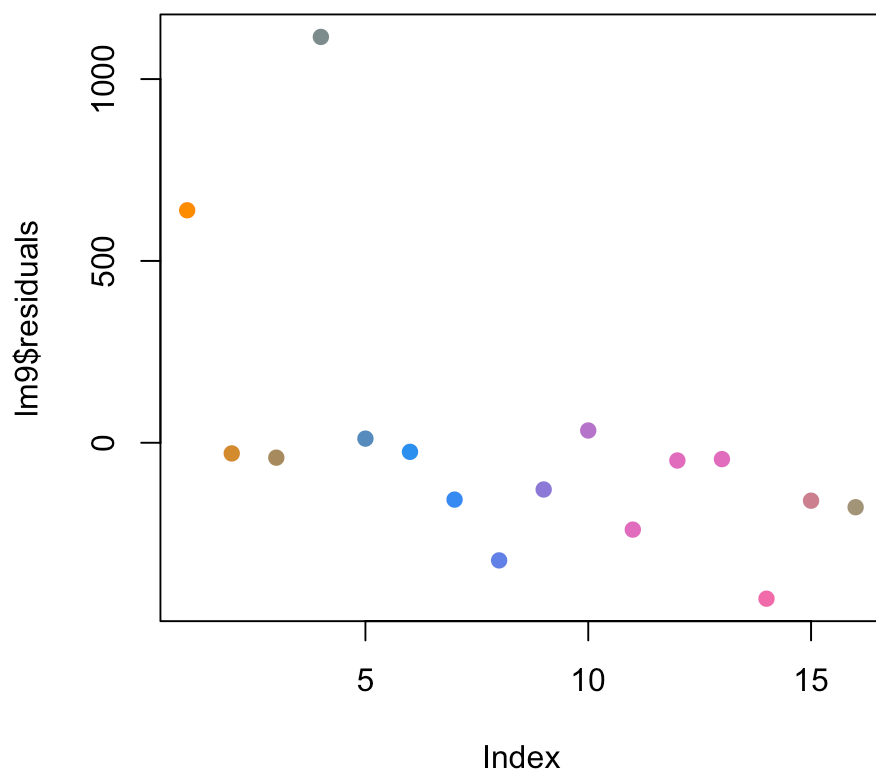
Be careful when two variables in your regression model are very highly correlated. This is called co-linearity and can lead to highly variable and uninterpretable results. R fails “gracefully” (i.e. doesn’t tell you) when this happens so you have to check by hand. This is also a problem if you fit too many variables for example.

```
lm8 = lm(gene1 ~ pdata$tissue.type + pdata$age)
tidy(lm8)
```

##		term	estimate	std.error	statistic
## 1		(Intercept)	10.40407714	NaN	NaN
## 2		pdata\$tissue.typeadrenal	-2.64252590	NaN	NaN
## 3		pdata\$tissue.typebrain	-2.64918963	NaN	NaN
## 4		pdata\$tissue.typebreast	-0.55076758	NaN	NaN
## 5		pdata\$tissue.typecolon	-0.90025140	NaN	NaN
## 6		pdata\$tissue.typeheart	-3.42679721	NaN	NaN
## 7		pdata\$tissue.typekidney	-0.76564122	NaN	NaN
## 8		pdata\$tissue.typeliver	0.52887576	NaN	NaN
## 9		pdata\$tissue.typelung	-0.73165179	NaN	NaN
## 10		pdata\$tissue.typelymphnode	-2.43829285	NaN	NaN
## 11		pdata\$tissue.typeovary	0.09875467	NaN	NaN
## 12		pdata\$tissue.typeprostate	-0.79305234	NaN	NaN
## 13		pdata\$tissue.typeskeletal_muscle	-4.27479412	NaN	NaN
## 14		pdata\$tissue.typetestes	0.47932985	NaN	NaN
## 15		pdata\$tissue.typethyroid	-0.46156263	NaN	NaN
## 16		pdata\$tissue.typewhite_blood_cell	-8.40407714	NaN	NaN
##		p.value			
## 1		NaN			
## 2		NaN			
## 3		NaN			
## 4		NaN			
## 5		NaN			
## 6		NaN			
## 7		NaN			
## 8		NaN			
## 9		NaN			
## 10		NaN			
## 11		NaN			
## 12		NaN			
## 13		NaN			
## 14		NaN			
## 15		NaN			
## 16		NaN			

Another good idea is to look for “patterns” in the residuals

```
colramp = colorRampPalette(1:4)(17)
lm9 = lm(edata[2,] ~ pdata$age)
plot(lm9$residuals,col=colramp[as.numeric(pdata$tissue.type)])
```



Notes and further reading

This is of course a ridiculously brief introduction to linear models. The main thing to keep in mind is that you should visually inspect the model fits, consider the scale you are measuring on, and evaluate whether the covariates are related in ways that are normal to you. Here are some places to learn a lot more about linear models

- Regression Models (Coursera) (<https://www.coursera.org/course/regmods>)
- Introduction to linear models and matrix algebra (EdX) (<http://online-learning.harvard.edu/course/introduction-linear-models-and-matrix-algebra>)

Session information

Here is the session information

```
devtools::session_info()
```

```

## setting value
## version R version 3.2.1 (2015-06-18)
## system x86_64, darwin10.8.0
## ui RStudio (0.99.447)
## language (EN)
## collate en_US.UTF-8
## tz America/New_York
##
## package * version date
## acepack 1.3-3.3 2014-11-24
## annotate 1.46.1 2015-07-11
## AnnotationDbi * 1.30.1 2015-04-26
## assertthat 0.1 2013-12-06
## BiasedUrn * 1.06.1 2013-12-29
## Biobase * 2.28.0 2015-04-17
## BiocGenerics * 0.14.0 2015-04-17
## BiocInstaller * 1.18.4 2015-07-22
## BiocParallel 1.2.20 2015-08-07
## biomaRt 2.24.0 2015-04-17
## Biostrings 2.36.3 2015-08-12
## bitops 1.0-6 2013-08-17
## bladderbatch * 1.6.0 2015-08-26
## broom * 0.3.7 2015-05-06
## caTools 1.17.1 2014-09-10
## cluster 2.0.3 2015-07-21
## colorspace 1.2-6 2015-03-11
## corpcor 1.6.8 2015-07-08
## curl 0.9.2 2015-08-08
## DBI * 0.3.1 2014-09-24
## dendextend * 1.1.0 2015-07-31
## DESeq2 * 1.8.1 2015-05-02
## devtools * 1.8.0 2015-05-09
## digest 0.6.8 2014-12-31
## dplyr * 0.4.3 2015-09-01
## edge * 2.1.0 2015-09-06
## evaluate 0.7.2 2015-08-13
## foreign 0.8-66 2015-08-19
## formatR 1.2 2015-04-21
## Formula * 1.2-1 2015-04-07
## futile.logger 1.4.1 2015-04-20
## futile.options 1.0.0 2010-04-06
## gdata 2.17.0 2015-07-04
## genefilter * 1.50.0 2015-04-17
## geneLenDataBase * 1.4.0 2015-09-06
## geneplotter 1.46.0 2015-04-17
## GenomeInfoDb * 1.4.2 2015-08-15
## GenomicAlignments 1.4.1 2015-04-24
## GenomicFeatures 1.20.2 2015-08-14
## GenomicRanges * 1.20.5 2015-06-09
## genstats * 0.1.02 2015-09-05
## ggplot2 * 1.0.1 2015-03-17
## git2r 0.11.0 2015-08-12

```

##	GO.db	3.1.2	2015-09-06
##	goseq	* 1.20.0	2015-04-17
##	gplots	* 2.17.0	2015-05-02
##	gridExtra	2.0.0	2015-07-14
##	gtable	0.1.2	2012-12-05
##	gtools	3.5.0	2015-05-29
##	highr	0.5	2015-04-21
##	HistData	* 0.7-5	2014-04-26
##	Hmisc	* 3.16-0	2015-04-30
##	htmltools	0.2.6	2014-09-08
##	httr	1.0.0	2015-06-25
##	IRanges	* 2.2.7	2015-08-09
##	KernSmooth	2.23-15	2015-06-29
##	knitr	* 1.11	2015-08-14
##	lambda.r	1.1.7	2015-03-20
##	lattice	* 0.20-33	2015-07-14
##	latticeExtra	0.6-26	2013-08-15
##	lazyeval	0.1.10	2015-01-02
##	limma	* 3.24.15	2015-08-06
##	lme4	1.1-9	2015-08-20
##	locfit	1.5-9.1	2013-04-20
##	magrittr	1.5	2014-11-22
##	MASS	* 7.3-43	2015-07-16
##	Matrix	* 1.2-2	2015-07-08
##	MatrixEQTL	* 2.1.1	2015-02-03
##	memoise	0.2.1	2014-04-22
##	mgcv	* 1.8-7	2015-07-23
##	minqa	1.2.4	2014-10-09
##	mnormt	1.5-3	2015-05-25
##	munsell	0.4.2	2013-07-11
##	nlme	* 3.1-122	2015-08-19
##	nloptr	1.0.4	2014-08-04
##	nnet	7.3-10	2015-06-29
##	org.Hs.eg.db	* 3.1.2	2015-07-17
##	plyr	1.8.3	2015-06-12
##	preprocessCore	* 1.30.0	2015-04-17
##	proto	0.3-10	2012-12-22
##	psych	1.5.6	2015-07-08
##	qvalue	* 2.0.0	2015-04-17
##	R6	2.1.1	2015-08-19
##	RColorBrewer	1.1-2	2014-12-07
##	Rcpp	* 0.12.0	2015-07-25
##	RcppArmadillo	* 0.5.400.2.0	2015-08-17
##	RCurl	1.95-4.7	2015-06-30
##	reshape2	1.4.1	2014-12-06
##	rmarkdown	0.7	2015-06-13
##	rpart	4.1-10	2015-06-29
##	Rsamtools	1.20.4	2015-06-01
##	RSkittleBrewer	* 1.1	2015-09-05
##	RSQLite	* 1.0.0	2014-10-25
##	rstudioapi	0.3.1	2015-04-07
##	rtracklayer	1.28.9	2015-08-19

##	rversions	1.0.2	2015-07-13
##	S4Vectors	* 0.6.5	2015-09-01
##	scales	0.3.0	2015-08-25
##	snm	1.16.0	2015-04-17
##	snpStats	* 1.18.0	2015-04-17
##	stringi	0.5-5	2015-06-29
##	stringr	1.0.0	2015-04-30
##	survival	* 2.38-3	2015-07-02
##	sva	* 3.14.0	2015-04-17
##	tidyr	0.2.0	2014-12-05
##	UsingR	* 2.0-5	2015-08-06
##	whisker	0.3-2	2013-04-28
##	XML	3.98-1.3	2015-06-30
##	xml2	0.1.2	2015-09-01
##	xtable	1.7-4	2014-09-12
##	XVector	0.8.0	2015-04-17
##	yaml	2.1.13	2014-06-12
##	zlibbioc	1.14.0	2015-04-17
##	source		
##	CRAN (R 3.2.0)		
##	Bioconductor		
##	Bioconductor		
##	CRAN (R 3.2.0)		
##	CRAN (R 3.2.0)		
##	Bioconductor		
##	Bioconductor		
##	Bioconductor		
##	Bioconductor		
##	Bioconductor		
##	Bioconductor		
##	CRAN (R 3.2.0)		
##	Bioconductor		
##	CRAN (R 3.2.0)		
##	CRAN (R 3.2.0)		
##	CRAN (R 3.2.1)		
##	CRAN (R 3.2.0)		
##	CRAN (R 3.2.1)		
##	CRAN (R 3.2.1)		
##	CRAN (R 3.2.1)		
##	CRAN (R 3.2.0)		
##	CRAN (R 3.2.1)		
##	Bioconductor		
##	CRAN (R 3.2.0)		
##	CRAN (R 3.2.0)		
##	CRAN (R 3.2.2)		
##	Github (jdstorey/edge@a1947b5)		
##	CRAN (R 3.2.2)		
##	CRAN (R 3.2.1)		
##	CRAN (R 3.2.0)		
##	CRAN (R 3.2.0)		
##	CRAN (R 3.2.0)		
##	CRAN (R 3.2.0)		
##	CRAN (R 3.2.0)		
##	CRAN (R 3.2.1)		

```
## Bioconductor
## Bioconductor
## Bioconductor
## Bioconductor
## Bioconductor
## Bioconductor
## local
## CRAN (R 3.2.0)
## CRAN (R 3.2.2)
## Bioconductor
## Bioconductor
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## CRAN (R 3.2.0)
## CRAN (R 3.2.0)
## CRAN (R 3.2.0)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## Bioconductor
## CRAN (R 3.2.1)
## CRAN (R 3.2.2)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## CRAN (R 3.2.0)
## CRAN (R 3.2.0)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## CRAN (R 3.2.0)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## Bioconductor
## CRAN (R 3.2.1)
## Bioconductor
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## Bioconductor
## CRAN (R 3.2.1)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
```

```
## CRAN (R 3.2.1)
## CRAN (R 3.2.1)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## CRAN (R 3.2.1)
## Bioconductor
## Github (alyssafranze/RSkittleBrewer@0a96a20)
## CRAN (R 3.2.0)
## CRAN (R 3.2.0)
## Bioconductor
## CRAN (R 3.2.1)
## Bioconductor
## CRAN (R 3.2.2)
## Bioconductor
## Bioconductor
## CRAN (R 3.2.1)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## Bioconductor
## CRAN (R 3.2.0)
## CRAN (R 3.2.2)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## CRAN (R 3.2.2)
## CRAN (R 3.2.0)
## Bioconductor
## CRAN (R 3.2.0)
## Bioconductor
```

It is also useful to compile the time the document was processed. This document was processed on: 2015-09-06.